Introduction aux Tests

Patrice Torguet

Université de Toulouse

Plan

- Introduction : pourquoi tester ?
- Types de tests
- Les tests unitaires
- Couverture des tests
- TDD
- Junit
- Doublures de tests
- Mockito
- Conclusion
- Références

Pourquoi tester ?

- Pour gagner du temps
- Pour valider le logiciel
- Pour valider les modifications du code
- Pour détecter les régressions

https://fr.wikibooks.org/wiki/Introduction_au_test_logiciel/Introduction

https://martinfowler.com/articles/practical-test-pyramid.html

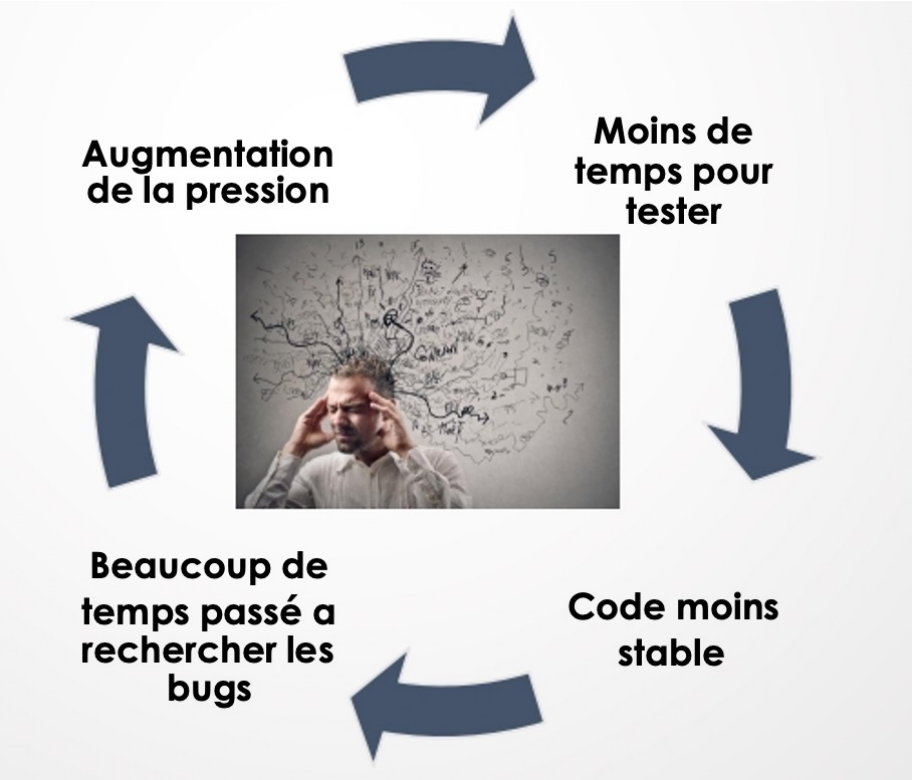
https://sandipsandilya.wordpress.com/2011/01/17/20-famous-software-disasters/

Mauvaises pratiques communes

- SOUVENT : Affichage en console
- PARFOIS : Traces des fonctions / programmes avec quelques cas
- RAREMENT : Utilisation d'un debugger

- Problèmes
  - Difficulté à isoler les éléments que vous voulez tester
  - Perte de lisibilité à cause des nombreux affichages
  - Plus généralement : manque de recul / hauteur !
  - Celui qui code est également celui qui teste !

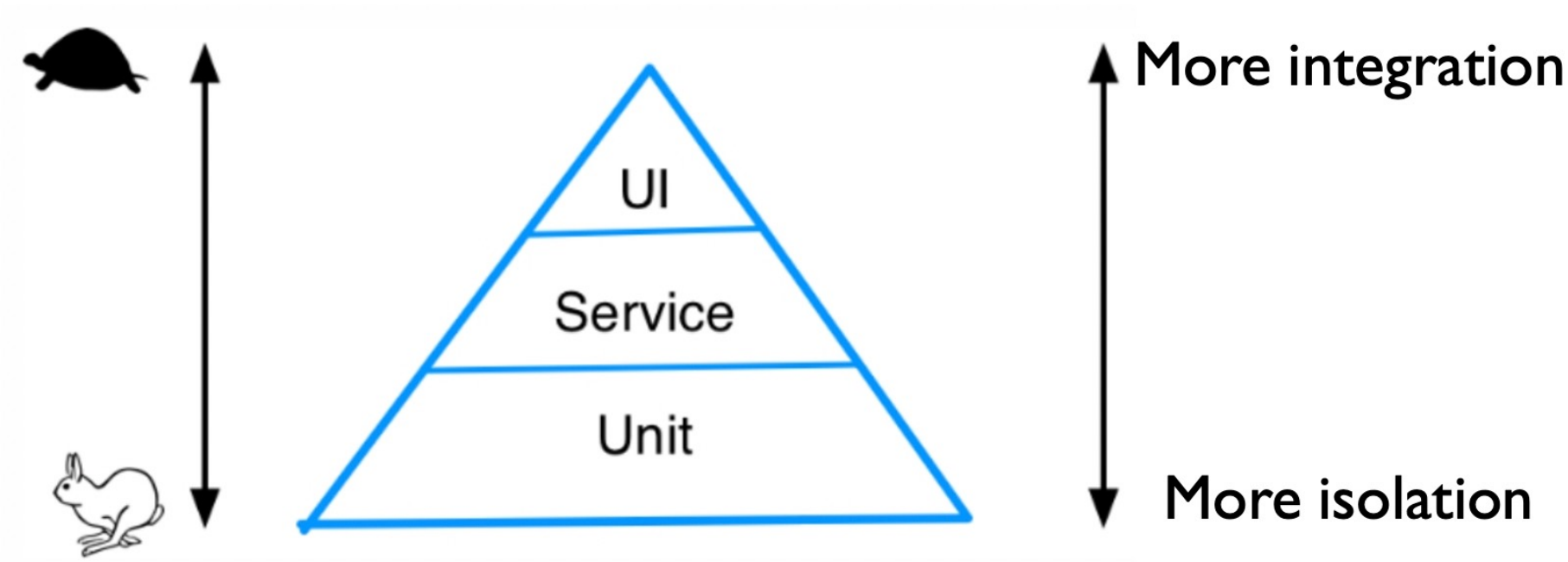
Paradoxe du test



Types de tests

- Tests unitaires
- Tests d'intégration
- Tests fonctionnels
- Tests de charge
- Autres : accessibilité, W3C compliance, utilisabilité, etc.

Pyramide des tests de Mike Cohn



Limites des tests

- Les tests ne garantissent pas le fonctionnement correct d’un logiciel
- L’évaluation est partielle
- Un test peut aussi contenir des bugs

Tests unitaires

- Isoler chaque « composant unitaire » d’un logiciel pour vérifier qu’il fonctionne comme il a été spécifié
- Définition d’un contrat que doit remplir le composant unitaire
- Ce contrat doit être exécutable pour que les tests soient automatisés
- En POO : l’unité est la classe

Un test unitaire doit être

- Isolé (indépendant des autres tests)
- Déterministe (c’est à dire : reproductible)
- Simple et rapide à coder et à exécuter
- Exécutable automatiquement
- Limité à un composant unitaire isolé

Principe d’un test

- Soit …
  - Construction d’un état initial du « monde »
- Quand …
  - Exécution du code testé
- Alors …
  - Vérification des résultats obtenus (assertions, métaphore de l’oracle)

Exemple d’oracle

```
public boolean testConcat() {
        String t1 = "Bonjour " ; // état initial
        String t2 = "Toto";
        String attendu = "Bonjour Toto"; // état attendu
        String arrivee = t1.concat(t2); // exécution du code testé
        return arrivee.equals(attendu); // vérification par l’oracle
}
```

Remarque : pour coder ce test, il n’est pas nécessaire de savoir comment est codée la méthode concat.

Exemples

- JUnit en Java pour tester du code Java
  - https://github.com/spring-projects/spring-framework/blob/main/spring-core/src/test/java/org/springframework/core/io/ClassPathResourceTests.java
- JUnit en Kotlin pour tester du code Kotlin
  - https://github.com/elaastic/elaastic-questions-server/blob/develop/src/test/kotlin/org/elaastic/questions/lti/LmsServiceIntegrationTest.kt
- Spock en Groovy pour tester du code Java
  - https://github.com/elaastic/elaastic-questions/blob/develop/test/integration/org/tsaap/assignments/AssignmentServiceIntegrationSpec.groovy

Que doit-on tester ?

- Une classe de test par classe du logiciel utilisée en production
- Tester les interfaces publiques de la classe
- Ne pas tester le code trivial

Cf. https://martinfowler.com/articles/practical-test-pyramid.html#WhatToTest de Ham Vocke

Quels cas doit-on tester ?

- Les cas nominaux => cas pour lesquels l’algo fonctionne
- Les cas d’erreur => permet de vérifier la gestion d’erreur
- Les cas limites => permet de vérifier la robustesse du code

Résultats des tests

- Succès : le test a réussi
- Echec : au moins une assertion est fausse
- Erreur : déclenchement d’une erreur non prévue

Couverture du code

- Quand savons-nous qu’on a suffisamment écrit de tests ?
- Le concept de « couverture du code » / « code coverage » permet d’avoir un indicateur quantitatif
- Attention : ce n’est pas qualitatif par contre

Coverage: org.elaastic.questions in elaastic-questions...

73% classes, 49% lines covered in package 'org.elaastic.questions.lti'

Element	Class, %	Method, %	Line, %
controller	75% (3/4)	50% (36/71)	28% (63/223)
oauth	25% (1/4)	9% (1/11)	13% (7/51)
LmsAssignment	100% (1/1)	85% (6/7)	87% (7/8)
LmsService	100% (1/1)	77% (14/18)	85% (49/57)
LmsUser	100% (1/1)	100%	
LmsUserAccountCreationService	100% (1/1)	90%	
LtiActivity	100% (1/1)	85%	
LtiConsumer	100% (1/1)	94%	
LtiUser	100% (1/1)	100%	

```
 * @param ltiLmsKey the lti consumer key
 * @param ltiUserId the user id coming from lti consumer
 * @return the lms user or null if not find
 */
fun findLmsUser(ltiLmsKey: String, ltiUserId: String): LmsUser? {
    val lms : LtiConsumer = ltiConsumerRepository.findById(ltiLmsKey).get()
    return lmsUserRepository.findByLmsUserIdAndAndLms(ltiUserId, lms)
}

/**
 * Get lms user based on lti launch information. Create a new one if required
 *
 * @param ltiUser the lti user built from consumer launch information
 * @return the lms user
 */
fun getLmsUser(ltiUser: LtiUser): LmsUser {
    val lms : LtiConsumer = ltiConsumerRepository.findById(ltiUser.lmsKey).get()
    var lmsUser : LmsUser? = lmsUserRepository.findByLmsUserIdAndAndLms(ltiUser.lmsUserId, lms)
    if (lmsUser == null) {
        lmsUserAccountCreationService.createUserFromLtiData(ltiUser).let { user ->
            lmsUser = createLmsUserFromLtiDataLmsAndUser(ltiUser.lmsUserId, lms, user)
        }
    }
    return lmsUser!!
}
```

CC BY NC SA

Conseils méthodologiques

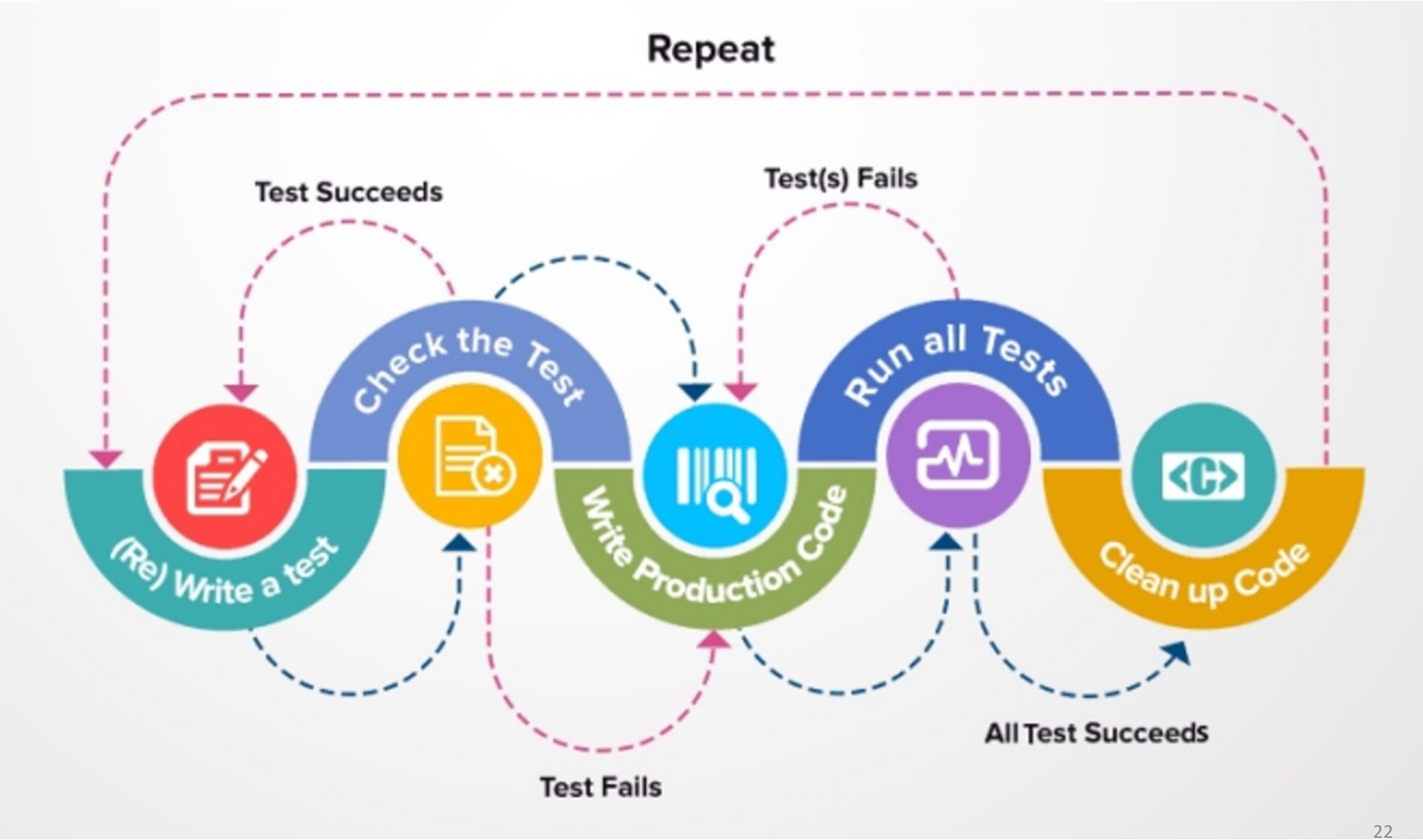
- Pour pouvoir écrire des tests unitaires
  - Les composants doivent être séparables (separation of concerns)
  - Les composants doivent être simples
- Si vous débuggez avec des « println », il vaut mieux écrire un test à la place
- Quand vous trouvez un bug, écrivez un test qui le caractérise

TDD : Test Driven Development

- Conceptualisé au départ par Erich Gamma et Kent Beck
- Puis livre de Kent Beck : Test Driven Development : By Example
- Utiliser des tests unitaires comme spécification du code
- Généralement intégré avec des méthodes agiles (eXtreme Programming, Scrum…)

Cycle de TDD

1. Écrire un test
2. Vérifier qu'il est en échec (car il n'y a pas encore de code fonctionnel)
3. Écrire juste assez de code pour que le test passe
4. Vérifier que le test passe
5. Réviser le code (refactoring) : améliorer le code et sa qualité tout en maintenant la même fonctionnalité


Repeat
Test Succeeds
Test(s) Fails
Check the Test
Run all Tests
(Re) Write a test
Write Production Code
Clean up Code
All Test Succeeds
Test Fails

Quelques avantages du TDD

- On utilise le logiciel avant même qu'il n'existe (on utilise l'interface du code avant de le coder)
- Diminue les erreurs de conception
- Construction simultanée du logiciel et de la suite de test de non-régression

JUnit

- Cadriciel pour les tests unitaires en Java
- Open Source : https://github.com/junit-team/junit
- Intégré dans les EDI Intellij, Eclipse, Netbeans mais aussi avec Maven
- Très utilisé dans l'industrie
- Version présentée ici : V4
- Une V5 existe on précisera quelques différences
- https://howtodoinjava.com/junit5/junit-5-vs-junit-4/

Rapports : exemple avec Intellij

Run: org.elaastic.questions in elaastic-questions...

Tests passed: 146 of 146 tests – 13 s 931 ms

questions (org.elaastic)	13 s 931 ms
AttachmentTest	946 ms
test dimension comparison()	536 ms
test attachement is a displayable image()	37 ms
test validation on valid attachment()	257 ms
test attachement is a displayable text()	37 ms
test attachement is not displayable()	8 ms
test validation on invalid attachment()	71 ms
AttachmentIntegrationTest	1 s 164 ms
testDetachAttachment()	363 ms
testSaveStatementAttachment()	42 ms
test the delete of an attachment and file system()	95 ms
test save an attachment with statement()	216 ms

```
09:56:38.956 [main] DEBUG org.jboss.logging
09:56:38.960 [main] INFO org.hibernate.vali
09:56:38.976 [main] DEBUG org.hibernate.val
09:56:38.977 [main] DEBUG org.hibernate.val
09:56:38.983 [main] DEBUG org.hibernate.val
09:56:38.985 [main] DEBUG org.hibernate.val
09:56:38.985 [main] DEBUG org.hibernate.val
09:56:38.986 [main] DEBUG org.hibernate.val
09:56:39.207 [main] DEBUG org.hibernate.val
09:56:39.221 [main] DEBUG org.hibernate.val
09:56:39.221 [main] DEBUG org.hibernate.val
09:56:39.221 [main] DEBUG org.hibernate.val
09:56:39.221 [main] DEBUG org.hibernate.val
09:56:39.221 [main] DEBUG org.hibernate.val
```

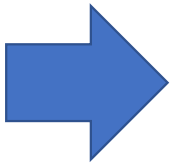
Terminologie JUnit

- Classe de Test : contient différents cas tests sur la même classe
- Méthode de Test : un cas test sur une méthode
- Assertion : expression dont on veut vérifier la véracité
- Fixture test : construit un état commun qui va être utilisé par plusieurs cas tests
- Suite de tests : permet de grouper des classes de test qui doivent être lancées ensemble

Classe de test

- Importer les classes et méthodes statiques
- ➨ import org.junit.Test; (en V5 : org.junit.jupiter.api.Test)
- ➨ import static org.junit.Assert.*;
  (en V5 : org.junit.jupiter.api.Assertions.*)

```
//Classe qui permet de manipuler un compteur
public class Compteur{
        int valeur;
        Compteur(){...}
        Compteur(int init ){...}
        int increment (){...}
        int decrement (){...}
        int getValeur (){...}
}
```



```
import org.junit.Test;
import static org.junit.Assert.*;

//Classe dédiée au test de la classe Compteur
public class CompteurTest {

}
```

Méthode de test

- Visibilité public
- Type de retour void
- Pas de paramètres
- Peut lever des exceptions
- Annotée @Test
- Utilise des instructions d'assertion

```
import org.junit.Test;
import static org.junit.Assert.*;


//Classe dédiée au test de la classe Compteur
public class CompteurTest {
        @Test
        public void testIncrement(){
        //La méthode increment() est testée ici

        }
        @Test
        public void testDecrement(){
        //La méthode decrement() est testée ici

        }
}
```

Principales Assertions

- Une assertion qui n'est pas vérifiée lance une exception

Instruction	**Description**
fail(message)	Permet de mettre le test en échec
assertTrue([message],condition)	Vérifie si la condition est vraie
assertEquals([message],expected,actual)	Vérifie si les valeurs sont égales
assertSame([message],expected,actual)	Vérifie si les deux références pointent sur le même objet
assertNotSame([message],expected,actual)	Vérifie si les deux références pointent sur des objets différents
assertNull([message],object)	Vérifie si l'objet est null
assertNotNull([message],object)	Vérifie si l'objet est non null

Méthode de test

```
import org.junit.Test;
import static org.junit.Assert.*;

//Classe dédiée au test de la classe Compteur
public class CompteurTest {
		@Test
		public void testIncrement(){
				int attendue;

				attendue = 1;
				Compteur c1 = new Compteur();
				c1.incrementer();
				assertTrue(c1.getValeur() == attendue);

				attendue = 11;
				Compteur c2 = new Compteur(10);
				c2.incrementer();
				assertEquals("Echec test 2",
c2.getValeur(), attendue);
		}
}
```

Valeur attendue

Création des objets pour le test

Appel de la méthode à tester

Vérification que le résultat correspond bien au résultat attendu.

Si ce n'est pas le cas:

1. Lance une *AssertionFailedError* et la méthode de test s'arrête
2. L'exécuteur de Test JUnit attrape cet objet et indique que la méthode a échoué
3. La méthode de test suivante est exécutée

Les méthodes assertX sont écrites en utilisant fail

```
import org.junit.Test;
import static org.junit.Assert.*;

//Classe dédiée au test de la classe Compteur
public class CompteurTest {
        @Test
        public void testIncrement(){
                int attendue;

                attendue = 1;
                Compteur c1 = new Compteur();
                c1.incrementer();
                assertTrue(c1.getValeur() == attendue);

                attendue = 11;
                Compteur c2 = new Compteur(10);
                c2.incrementer();
                assertEquals("Echec test 2",
c2.getValeur(), attendue);
        }
}
```

```
attendue = 1;
Compteur c1 = new Compteur();
c1.incrementer();
if(c1.getValeur() != attendue){
        fail("ECHEC DU TEST");
}
```

Fixtures

Plusieurs méthodes de tests

```
import org.junit.Test;
import static org.junit.Assert.*;
public class CompteurTest {
        @Test
        public void testIncrement(){
                Compteur c = new Compteur(10);
                c.incrementer();
                int attendu = 11;
                assertTrue(c.getValeur() == attendu);

        }
        @Test
        public void testDecrement(){
                Compteur c = new Compteur(10);
                c.decrementer();
                int attendu = 9;
                assertTrue(c.getValeur() == attendu);
        }
}
```

Partie commune à chaque test !

Fixtures

Plusieurs méthodes de tests

```
import org.junit.Test;
import static org.junit.Assert.*;
public class CompteurTest {
        @Test
        public void testIncrement(){
                Compteur c = new Compteur(10);
                c.incrementer();
                int attendu = 11;
                assertTrue(c.getValeur() == attendu);

        }
        @Test
        public void testDecrement(){
                Compteur c = new Compteur(10);
                c.decrementer();
                int attendu = 9;
                assertTrue(c.getValeur() == attendu);
        }
}
```

```
//Attributs
private Compteur c;
```

```
//Initialiser le compteur avant chaque test
@Before
public void setUp(){
        c = new Compteur(10);
}
```

@BeforeEach en V5

Annotations

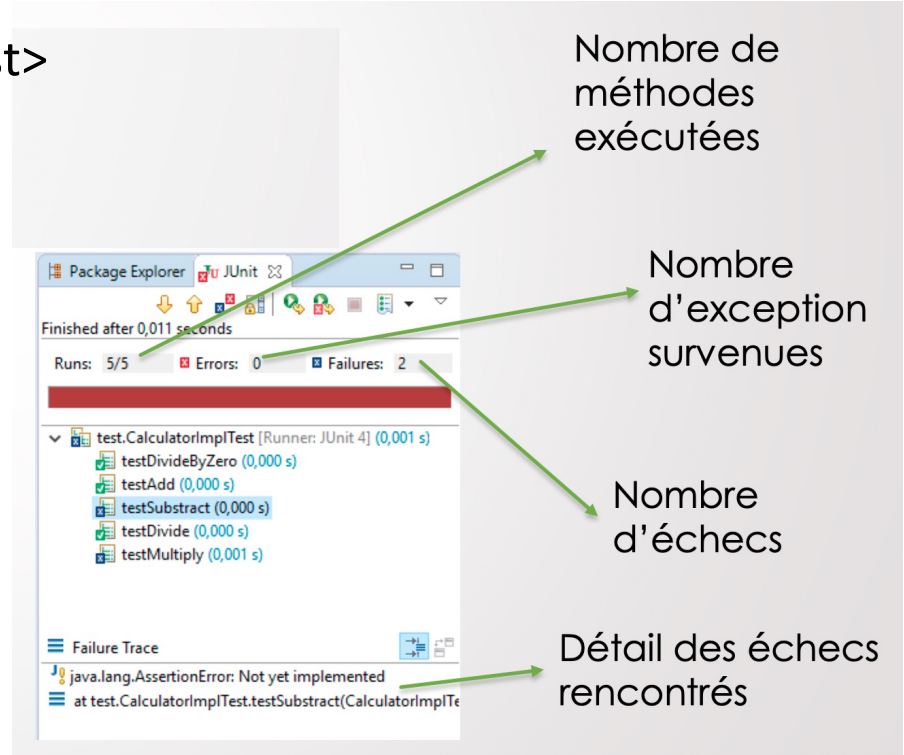
Annotation (V4)	Description
@Test	Définit une méthode de test
@Before	Méthode exécutée avant chaque test
@After	Méthode exécutée après chaque test
@BeforeClass	Méthode static exécutée avant le premier test
@AfterClass	Méthode static exécutée après le dernier test

Annotations

- Ordre d’exécution
  1. La méthode annotée @BeforeClass (@BeforeAll pour Junit5)
  2. Pour chaque méthode annotée @Test (ordre indéterminé)
     1. Les méthodes annotées @Before (@BeforeEach pour Junit5)
     2. La méthode annotée @Test
     3. Les méthodes annotées @After (@AfterEach pour Junit5)
  3. La méthode annotée @AfterClass (@AfterAll pour Junit5)

Lancement des tests

- En ligne de commande
  - java org.junit.runner.JUnitCore <maClasseTest>
- Depuis du code Java
  - junit.textui.TestRunner.run(<maClasseTest>);
- Depuis un IDE
  - Eclipse : Run > Run As > Junit Test



Gestion des exceptions

```
public class CompteurTest {
        private Compteur c;

        //La méthode décrémenter doit lever une CompteurZeroException si le compteur est à zéro

        @Test (expected=CompteurZeroException .class)
        public void testDecrementAPartirdeZero(){
                c = new Compteur(0);
                c.decrementer();
        }
}
```

En V5 :

```
@Test
void testDecrementAPartirDeZero() {
    c = new Compteur( valeur: 0);
    Assertions.assertThrows(CompteurZeroException.class, () -> {
        c.decrement(); // génère CompteurZeroException
    });
}
```

Gestion du temps d'exécution

```
public class CompteurTest {
        private Compteur c;

        //La méthode doit échouer si le temps d'exécution dépasse un seuil

        @Test (timeout=10000)
        public void testIncrementMultiple(){
                c = new Compteur(0);
                c.incrementer(1000000);
        }
}
```

En V5 :

```
@Test
@Timeout(value = 100, unit = TimeUnit.MILLISECONDS)
void failsIfExecutionTimeExceeds100Milliseconds() {
    // fails if execution time exceeds 100 milliseconds
}
```

Suite de tests

```
import org.junit.runners.Suite;
import org.junit.runner.RunWith;

@RunWith(Suite.class)
@Suite.SuiteClasses({TestClass1.class, TestClass2.class})
public class TestSuite {
  // rien ici
}
```

En V5 :

```
import org.junit.platform.suite.api.IncludeClassNamePatterns;
import org.junit.platform.suite.api.SelectPackages;
import org.junit.platform.suite.api.Suite;
import org.junit.platform.suite.api.SuiteDisplayName;

@Suite
@SuiteDisplayName("JUnit Platform Suite Demo")
@SelectPackages("example")
@IncludeClassNamePatterns(".*Tests")
class SuiteDemo {
}
```

Exemple Simple

- Une interface pile : Stack
- Une première implantation : SimpleStack
- Une classe de test unitaire : SimpleStackTest

L’interface Stack

```java
public interface Stack {

 /**
 * Tests if this stack is empty
 */
 public boolean isEmpty();

 /**
 * Returns the number of items in this stack.
 */
 public int getSize();

 /**
 * Pushes an item onto the top of this stack.
 * null item is allowed.
 */
 public void push(Item item);

 /**
 * Looks at the object at the top of this stack without removing it from the stack.
 */
 public Item peek() throws EmptyStackException;

 /**
 * Removes the object at the top of this stack and returns
 * that object as the value of this function.
 * @throws EmptyStackException if this stack is empty.
 */
 public Item pop() throws EmptyStackException;

 }
```

La classe SimpleStack

```java
public class SimpleStack implements Stack {

    @Override
    public boolean isEmpty() {
    // TODO Auto-generated method stub
    return false;
    }

    @Override
    public int getSize() {
    // TODO Auto-generated method stub
    return 0;
    }

    @Override
    public void push(Item item) {
    // TODO Auto-generated method stub
    }

    @Override
    public Item peek() throws EmptyStackException {
    // TODO Auto-generated method stub
    return null;
    }

    @Override
    public Item pop() throws EmptyStackException {
    // TODO Auto-generated method stub
    return null;
    }
}
```

La classe SimpleStackTest

```java
import static org.junit.Assert.*;
import org.junit.Test;

public class SimpleStackTest { // Test class

@Test
public void testCreateEmptyStack() { // Test case

// When a freshly stack is created
Stack stack = new SimpleStack();

// Then… (oracle)
assertTrue("A new stack should be empty", stack.isEmpty());
assertEquals("A new stack has no element", 0, stack.getSize());
}

@Test
public void testPush() throws EmptyStackException {

// Given an empty stack and an item
Stack stack = new SimpleStack();
Item item = new SimpleItem();

// When the item is pushed in the stack
stack.push(item);

// Then…
assertFalse("The stack must be not empty", stack.isEmpty());
assertEquals("The stack constains 1 item", 1, stack.getSize());
assertSame("The pushed item is on top of the stack", item, stack.peek());
}
}
```

La classe SimpleStackTest

```
@Test(expected = EmptyStackException.class)
  public void testPopOnEmptyStack()  throws EmptyStackException {
  // Given an empty stack
  Stack stack = new SimpleStack();

  // When we « pop » the stack
  stack.pop(); // should throws an EmptyStackException.
  }
```

Conseils pratiques

- Une classe test par classe
  - **<NomClasse>Test**
- Une ou plusieurs méthodes test par méthode
  - **test<NomMethode>[<cas>][<attendu>]**
- Les classes de test doivent être séparées des classes du logiciel
  - Exemple de Maven : src/main/* et src/test/*

Les doublures de test

- Problème : une classe utilise une autre classe qui n'existe pas encore
- Exemple : projet avec plusieurs développeurs
- Solution : écrire une version simplifiée de la classe juste pour les tests
- Plusieurs versions :
  - Bouchon/Substitut : classe écrite à la main qui correspond à une version simplifiée de la classe. Soit spécifique à un test (bouchon) soit plus générale (substitut)
  - Simulacre (mock) : objet créé automatiquement par un outil. On peut préciser le comportement des méthodes avec des assertions de type : quand / alors comme pour les oracles de test

Exemple de Bouchon/Substitut : pile à deux valeurs

- private class StubStack implements Stack {

```
    private int nbVal = 0;
    private double val1 = -1;
    private double val2 = -1;
    …
    public void push(double item) {
        switch(nbVal) {
            case 0:
                val1 = item;
                nbVal=1;
                break;
            case 1:
                val2 = val1;
                val1 = item;
                nbVal=2;
                break;
        }
    }
    …
```

Mockito

- Outil pour générer des objets simulacres (mock objects)
- mock sert pour créer les objets en précisant l'interface ou la classe à simuler
- when/thenReturn/thenThrow sert pour définir le comportement
- verify permet d'espionner les appels aux méthodes
- Il existe aussi spy pour espionner les appels sur de vrais objets

- https://site.mockito.org/

Exemples avec Mockito

- Création d'un objet simulacre :

```
List<Double> list = (List<Double>)mock(List.class);
```

- Spécification du comportement

```
when(list.isEmpty()).thenReturn(true).thenReturn(false);
when(stack.pop()).thenReturn(1.0).thenThrow(new EmptyStackException());
```

- Pour les méthodes void

```
doThrow(new DivideByZeroException()).when(calc).divide();
```

- Vérifications :

```
verify(list).isEmpty();
verify(list).add(0, 1.0);
verify(pile1, times(2)).push(2);
```

Conclusion

- Plus on teste et moins on perdra de temps à debugger
- Déployer un logiciel non (ou mal) testé fait perdre en crédibilité
- Les tests unitaires sont un bon début mais ne suffisent pas
- En Intégration Continue les tests sont appliqués automatiquement lors de tout push git

Références

- Ce cours est basé sur les cours sur les tests de
  - Franck Sylvestre francksilvestre.github.io
  - Erick Stattner www.erickstattner.com

- https://fr.wikibooks.org/wiki/Introduction_au_test_logiciel
- https://martinfowler.com/articles/practical-test-pyramid.html